

COS341 - SPL Compiler Phase 1

User Manual

Project Phase: Frontend Phase 1

Executable File: group-30.jar

1. Group Information – Group 30

Member Role	Full Name	Student Number
Speaker	Karabo Nkomo	u24865169
Group Member	Sente Mngomezulu	u24874478
Group Member	Hayley Booysen	u24868346

2. System Overview

group-30.jar is a combined Lexical and Syntax Analyzer for Student Programming Language (SPL). It executes as a standalone Java executable containing a robust LL(1) recursive-descent frontend.

Operations Overview

1. **Input:** Reads an uncompiled ASCII source file containing SPL code
2. **Lexical Analysis:** Scans the stream for valid tokens, enforcing strict specification constraints. It programmatically appends the synthetic end-of-file marker \$ (which is not expected inside the source text).
3. **Syntax Analysis:** Parses tokens into an Abstract Syntax Tree (AST).
4. **Output:**
 - **On Success:** Generates a complete XML syntax tree written directly to tree.xml in the current working directory.
 - **On Syntax/Lexical Error:** Prints a descriptive error message indicating the exact **Line Number** and token context where validation failed.

3. Instructions for Execution

To execute the SPL compiler, ensure that a Java Runtime Environment (JRE 11 or higher) is installed on the testing system. Open a command prompt or terminal in the directory containing the submission files.

The software accepts the target input file through **two mechanisms**:

Mode 1: Command-Line Argument

You can pass the input file path directly as a command-line parameter upon execution.

Usage:

```
java -jar group-30.jar <path_to_input_file>
```

Mode 2: Interactive Prompt (Fallback)

If executed without command-line arguments, the program will prompt the user in the console to enter the file path manually.

Usage:

```
java -jar group-30.jar
```

Console Execution Flow:

```
Plaintext
=====
    SPL Compiler Phase 1 (Group XX)
=====
Enter input file path: SPL.txt
--- Reading SPL.txt ---
[Lexer Success] Generated 41 tokens.
--- Parsing ---
[Parser Success] AST Root constructed: <SPL_PROG>
--- Generating XML ---
[XML Success] Syntax tree written to tree.xml
```

4. Output Description

Successful Parse (`tree.xml`)

When the SPL file contains syntactically correct code, the program creates **tree.xml** in the exact directory from which the **group-30.jar** is executed. The XML file is structured with a single `<tree>` root element that encloses all syntax tree nodes. Inside this root, each node is listed sequentially with its unique ID, non-terminal/terminal contents, parent links, and child relationships:

Example:

```
<tree>
<node>
  <id>1</id>
  <contents>SPL_PROG</contents>
  <children>
    <child>2</child>
  </children>
</node>
<node>
  <id>2</id>
  <contents>P</contents>
  <parent>1</parent>
  <children>
    <child>3</child>
    <child>8</child>
    <child>40</child>
  </children>
</node>
</tree>
```

Error Reporting

If a syntax error or lexical violation is encountered, execution halts immediately and outputs the error details without generating an invalid XML file:

Plaintext

--- Reading SPL-Err.txt ---

[Lexer Success] Generated 37 tokens.

--- Parsing ---

[Parser Error] Syntax Error at line 7: Expected 'else' after 'then' block in branch.